AI Stalemate



THE LEAD DEVELOPER FIELD GUIDE

Explain the concerns. Work out the next step with your PM.

Nic Payne

For you, the lead developer

OPEN YOUR LEAD DEVELOPER LAB >

START WITH A REAL SITUATION. DECIDE WHAT HAPPENS NEXT.

WORKING EDITION / REBEL-LEADER.COM

Your PM wants to try AI. You have questions.

Your PM has an AI idea they want to explore. You can see the checks, review, and ongoing work it would need. Let's make those clear and hear what they hope to improve, so you can decide what to do together.

What are you hoping to improve?

Ask your PM what they want to make better. Perhaps support staff copy repair requests into two systems and customers have to call again when one is missed. That gives you a specific problem to discuss.

What is your colleague worried about?

Show your PM the concern: the app must prevent one customer from seeing another's records, and someone needs time to review the code. Say what you would need to check and who could help.

What have you assumed about each other?

'They asked me to try it by Friday' describes what happened. 'They don't care about quality' is an explanation you may have added. What could you ask to understand what's behind the request?

What could you work out together?

You might agree to try something small. You might need permission, advice, or more information first. You might decide this use isn't worth pursuing. All of those can be useful answers.

The aim is to understand the opportunity and the concerns well enough to decide what to do next.

Reading and reflection: see the companion at the end of this Guide >

Give them something they can picture.

Let's start with something your colleague can picture. If this went well, whose work or experience would get better, and how?

Describe an ordinary day

Our support team copies every repair request into two systems. Sometimes they miss one, and a customer has to call again. We want to stop making customers chase work they have already requested.

Explain why you care

Saving time matters because people could use it for something else: helping a customer, investigating a difficult fault, or finishing work that keeps being postponed. Say what you hope that time would make possible.

Leave room for their answer

Ask what they would add or change. They may see a simpler way to help, or a part you have missed. Give their answer room to change the idea.

Think beyond this one task

You might learn a way of working the team can trust and teach to others. That could give people more confidence and room to grow. Treat it as a possibility to explore together, not a reward you can promise.

> Try: 'I can see why you want to fix this. I'm worried about who could see the customer records. Can we work through that and make time for someone to check the code before we try it?'

Reading and reflection: see the companion at the end of this Guide >

What would need to change?

A concern is easier to work with when you know what it refers to. Ask for an example. Sometimes you can change the plan. Sometimes the answer is no, and you need to understand why.

‘The code isn’t good enough.’

Which checks does it fail? Would a smaller task, a good example, or clearer instructions help? Keep the checks you normally require before accepting work.

‘Reviewing it takes longer.’

Ask where the extra work appears. Count the author’s checking, someone else’s review, and the fixes. A faster first draft can still take more work to finish.

‘We can’t put that data in the tool.’

Find out which data and which tool are involved. Ask the person responsible for permission. A test with made-up records may be possible, but check before starting.

‘It doesn’t know our code.’

Ask what it is missing. A small task with a relevant example may be worth trying. If it needs access you can’t give it, change the task or stop.

‘We don’t have time.’

Who would do the work and who would review it? What would they put off? Get that choice agreed. Quietly adding an experiment to someone’s evening isn’t a plan.

‘We tried this already.’

Ask what they tried and what happened. Use what they learned. Another experiment needs a reason: what is different, and what do you still need to find out?

GitHub: Copilot best practices

Rahul Garg: reducing AI development friction

Try: ‘Can you show me where it goes wrong? I want to understand whether we can change the idea or should leave this use alone.’

Reading and reflection: see the companion at the end of this Guide >

Before you talk with your product manager.

Take five or ten minutes before the conversation. You don't need a perfect argument. You need to know what matters to you and what you still need to ask.

What happened?

Describe what was said or done, using words the other person would recognize.

What am I assuming?

Notice the motive you have assigned to them. Ask about it rather than presenting it as a fact.

Why does this matter to me?

Describe the checks, review, and maintenance you would take responsibility for. Include what could be useful about the idea as well as what worries you.

What have I actually seen?

Separate something you observed from something you expect. Both can matter, but they are different kinds of information.

What would change my mind?

Name a result or explanation that would make you reconsider. Leave room for a better idea.

What do I still need to ask?

Write the question you haven't answered by yourself. That's a good place to start the conversation.

Private notes stay only in this open tab. They aren't saved or included in shared documents, and they disappear when you refresh or leave. You can also reflect without typing.

Reading and reflection: see the companion at the end of this Guide >

Ask what is behind your PM's request.

Your PM may have heard something from customers or leadership that you haven't. Ask what they want to improve and why it matters now. Then explain what you would need to do the work.

Who would this help?

Describe the people and the work. In our fictional example, support staff copy repair requests into two systems. An app might remove that repeated work and reduce missed requests.

Where did the number come from?

If someone says this could be worth $500,000 a week, ask how they worked it out. Is that money currently being lost, money they hope to earn, or time they expect to save? Say who supplied the estimate and who will check it.

What could the team learn?

People may become better at checking generated code or teaching others where it helps. Ask what would make the work worthwhile for them too. Don't promise promotion or rewards you can't control.

What would it take to find out?

Ask what help, permission, training, or time is missing. Take that specific request to the person who can approve it. Name what other work would wait.

Try: 'Here's what I've heard from the customer. I haven't checked this figure yet. What else would you need to know before we decide whether to try it?'

Reading and reflection: see the companion at the end of this Guide >

Who should be able to see this?

Imagine a customer opening the app and seeing another customer's repair history. Before connecting real systems, the people responsible need to agree who may see and change each kind of record—and verify that the software enforces it.

Decide who may do what

Name the users or groups, the records they may see, and whether they may change them. Decide who grants and removes access, including when someone changes jobs or leaves.

Check more than the sign-in screen

Signing in tells the system who someone is. It must also check what they are allowed to do. An API is a connection software uses to request data or actions; those requests need permission checks too. Hiding a button isn't enough.

Limit what the AI can do

Say which code, data, and system it may use. Can it only read, or can it make changes, run commands, or release software? Agree who approves those actions and who can undo a mistake.

Ask the right people to verify it

The lead developer and the people responsible for security and data need to define and check the rules. Record their names, the checks, and where the results can be found. Filling in this form doesn't grant permission.

Microsoft: app roles and groups

Microsoft: verify API scopes and roles

OWASP: authorization checks

If nobody has agreed or tested the permissions, resolve that first. A test using made-up records may help, but it also needs approval.

Reading and reflection: see the companion at the end of this Guide >

Can the team check the work it will be responsible for?

AI can produce code that looks plausible. Someone still needs to understand it, check it, and know what to do when it fails. Work out what help the team needs before giving it that responsibility.

Ask people to show their working

Can the developer explain the change, test who can access the data, inspect the code changes, and investigate a failure? Can they explain how to release it and restore the previous version? Find out by working through a small example together.

Make time for an experienced reviewer

Pair a developer with someone who can check this kind of work. Ask them to explain how they know the result is correct. Agree time for that conversation as part of the work.

Practise the whole change

Agree what the finished work must do. Keep a record of code changes, have someone review them, run the required tests, and practise releasing and undoing the change.

Keep the first task small

Choose a piece of work people can understand and check. Record what the team needs to learn before taking on more. Don't assume that writing more code means the team can safely maintain more software.

NIST: Secure Software Development Framework

GitHub: responsible use of Copilot agents

DORA: working in small batches

> 'Prepare first' is a useful answer when it names actual work: who will pair with whom, what they will practise, and what they need to demonstrate before starting.

Reading and reflection: see the companion at the end of this Guide >

What would be worth finding out?

Plan the experiment with your product manager. Explain which task is suitable, what needs checking, and how much review time you need. Ask them to help agree what other work will wait.

Say what you expect and why

For example: we think AI could help write permission tests faster because many of them are similar. That's a prediction. We haven't shown it yet.

Choose the task and the comparison

Name exactly what you'll try and what you'll leave out. Choose similar work done without AI so the comparison is useful. Note differences in task size, experience, or required checks.

Check permission

Record the approved tool and version, the code and data it may use, and who confirmed permission. Permission for this experiment doesn't cover live customer systems or wider use.

Agree the time and checks

Include time to prepare, write, check, review, and fix the work. Decide what must pass before you accept it. Agree what other work will wait and who can approve that change.

Decide what would count as useful

Write your prediction separately from the smallest improvement that would make another use worthwhile. Agree when to stop—for example, if a permission check fails or you use up the review time.

Give the plan to the people doing it

Name the author, reviewer, person who can approve the result, and review date. Confirm they have agreed. If something is missing, pause and name who will resolve it.

The idea of choosing how much time to invest draws on Ryan Singer's Shape Up (basecamp.com/shapeup). An experiment that never started has no result.

Reading and reflection: see the companion at the end of this Guide >

Did the faster draft save any time?

Count your work and the reviewer's work, including checks and fixes. Two people working for 30 minutes use 60 person-minutes. Give your PM the full picture of time and quality.

Work	Without AI	With AI
Preparation	10 min	15 min
Writing the first version	45 min	12 min
Author checking and testing	15 min	25 min
Someone else reviewing	15 min	30 min
Fixes	5 min	15 min
Later follow-up	0 min	0 min
Total working minutes	90 min	97 min

A separate fictional example

These numbers describe one small task writing software tests. The finished tests passed the required checks after fixes. This is separate from the preparation example in the Lab.

What happened?

AI cut drafting by 33 minutes. Checking, review, and fixes used 40 more. The whole task therefore took seven extra minutes. The reviewer spent 30 minutes, twice the agreed 15-minute limit; that limit was exceeded and needs to be addressed before another experiment.

What else do you need to know?

Record failed checks, later defects, tool costs, and anything that makes the tasks hard to compare. Ask the author and reviewer how the work felt. One task can't tell you what will happen everywhere.

Enter 0 when no time was spent. Leave a box blank when you don't know. A partial total can't show a saving.

Reading and reflection: see the companion at the end of this Guide >

What did we learn, and what should we do next?

Let's compare what happened with what you hoped to find. It's useful to learn that this task needs a different approach, or that this use of AI isn't worth continuing.

CONTINUE

The result supports using AI for this task under the agreed conditions. Say where that permission ends and when you'll check again.

REVISE

Something looks useful, but the task, instructions, or review needs to change. Describe the change before asking people to try again.

STOP

The result doesn't justify this use, or an agreed limit was broken. Explain why you're stopping and keep what you learned.

PAUSE

You're missing permission, information, time, or a person who needs to be involved. Name what is missing, who will resolve it, and when you'll return. You can choose this before an experiment.

Write what you decided, why, who will do what next, and when. If you haven't agreed, keep the record marked as a proposal and say what still needs deciding.

Reading and reflection: see the companion at the end of this Guide >

The next step is preparation.

Fictional case: your PM wants to help support staff stop copying repair requests between two systems. You see questions about who can see the customer records and whether the team can check and release the code safely.

What the PM knows

The sales director says reducing the delay could create $500,000 of value a week. Nobody has checked the calculation. The PM will ask Finance what the figure includes before treating it as a fact.

What the lead needs to check

Staff should see only the customer records their jobs require. The team needs tests that prove forbidden access is blocked, including direct requests to the app's API—the connection other software uses.

What they propose

Prepare first. The data owner will decide who may see which records. The engineering manager will pair developers with an experienced reviewer to practise checking, releasing, and undoing a change.

What they record

PAUSE this use for now. Check progress next Tuesday. No experiment or connection to live customer data is approved. A later test with made-up records would need its own permission.

What they tell leadership

'We can see why this could help. Before we try it, we need to agree who can see the customer records and check that the team can release a change safely. Can we make time for that preparation? We'll review it next Tuesday.'

They have a proposal someone can answer, with specific work and people responsible. They haven't measured an AI productivity gain.

Reading and reflection: see the companion at the end of this Guide >

Bring one real conversation.

Ready to try this with your own situation? The Lab helps you work through the questions and prepare something to talk over with your colleague. Start where it helps; you don't need to fill every box.

1. Before you talk

Write private notes, or think through the questions without typing. What matters to you, what are you assuming, and what do you still need to ask?

2. Conversation

Share what you want to improve and ask about the concerns. Choose what to do next. Preview Next Step Together, check it, then copy, download, or print it. Mark a proposal as a proposal.

3. Experiment, if useful

Agree the task, time, checks, permissions, people, and review date. Read through the Experiment Plan before starting. You can skip this step if your next move doesn't involve an experiment.

4. Results

Return after the experiment. Enter all the time spent, the quality checks, and what happened. Preview the Results Record. Leave unknown values blank.

5. Next step

Write the decision and the reason. Name who will do what next and when you'll check again. Preview the Decision Brief and review it before sharing.

Shared drafts are saved in this browser on this device. Download work you need to keep before changing devices or clearing browser data. Private notes are never saved. Load Examples offers fictional answers; if the step contains work, it asks before replacing it.

Reading and reflection: see the companion at the end of this Guide >

The thinking behind the tool.

Holistic Shaping asks what work is doing to people, as well as what people are doing at work. These short notes connect the Guide to Nic's teaching and a few practical sources. Read what helps with the situation in front of you. Links open the original resources.

Two maps you can return to

Cultural Orbital Mechanics
Nic Payne / Holistic Shaping · Framework resource

How identity, relationships, and shared vision give rise to culture—and how that culture shapes people in return.

Holistic Impact Depth
Nic Payne / Holistic Shaping · Framework resource

Use the tree and five areas of health to look at what everyday work is doing to people.

Back in the Guide

CHAPTER 01

Two people are being shaped by this conversation.

You may leave an AI discussion feeling heard, dismissed, more curious, or more certain that the other person is the problem. That matters beyond the next experiment. Ask what this repeated argument is teaching each of you about your place on the team.

Worth asking: What would help both of you take part without having to defend your worth?

The Hollow Crown: Why Our Leadership Narratives Have Failed
Nic Payne · Article

CHAPTER 02

Let their answer change the future.

"Wouldn't it be cool if…"

Nic Payne · A future you can imagine together

In Nic's orbital model, vision grows through relationship. Bring your idea for what could be better and invite your colleague to change it. An invitation to help shape the future needs room for an answer you didn't expect.

Worth asking: Which part of your proposal are you willing to rethink together?

Cultural Orbital Mechanics
Nic Payne / Holistic Shaping · Framework resource

The Lie We've Normalized: "Leadership Is Influencing Behavior"
Nic Payne · Article

A future you can imagine together
Nic Payne · Substack Note

CHAPTER 04

Meet the person behind the position.

Before opening the conversation, notice what you are listening for. Are you trying to catch a weak argument, or understand the person who has to live with this decision? Nic's bowl metaphor starts with the attention we bring. You are part of the relationship being shaped, too.

Worth asking: What might you hear if you stop preparing your reply for a moment?

Leadership as bowl, water, and current
Nic Payne · Video · start at 4:19

Every leader needs contact lenses
Nic Payne · Substack Note

The Mirror Move
Nic Payne / Rebel Leaders · Codex practice

CHAPTER 07

What will this way of working do to the team?

HID asks what our way of working does to people across five areas of health. Here, look beyond the code: room to learn the craft, confidence to question a result, a real say in the plan, work people can stand behind, and time to recover. Faster output can hide costs in any of those places.

Worth asking: What would you change so people become more capable rather than quietly carrying more?

Holistic Impact Depth
Nic Payne / Holistic Shaping · Framework resource

When Mattering Isn't Enough
Nic Payne · Article

CHAPTER 08

Make the investment explicit.

Ryan Singer's Shape Up separates the problem worth solving from the amount of time you are willing to invest. That distinction can help you describe this experiment. It does not turn an untested idea into an approved production change.

Worth asking: What are you willing to spend to answer this question, including review?

Shape Up — Chapter 6: Write the Pitch
Ryan Singer · Book chapter

CHAPTER 09

Keep the evidence in its setting.

METR's early-2025 study and its 2026 update show why an AI productivity headline needs context. Teams, tasks, tools, and the way work is selected matter. Use the studies to sharpen your questions; use your own records to decide what happened here.

Worth asking: What did your comparison count, and whose work could it have missed?

We are Changing our Developer Productivity Experiment Design
Joel Becker, Nate Rush, Tom Cunningham, David Rein, Khalid Mahamud / METR · Research update

Early-2025 developer productivity study
METR · Research paper

CHAPTER 10

Keep what you learned together.

Record the reason for continuing, changing, pausing, or stopping. Also ask what changed between you: can people raise a concern sooner, explain their judgment, or offer a better idea? Those experiences help shape the next conversation and, over time, what feels normal here.

Worth asking: What should the next person be able to understand without repeating this argument?

Cultural Orbital Mechanics
Nic Payne / Holistic Shaping · Framework resource

Do Your Replacement a Favor
Nic Payne · Article

Explore Nic's articles, video, and the complete reading library.

The notes apply the teaching to these situations. They are not quotations unless marked. The health reflection is optional; you do not need to record or share personal information.